

Assignment-2

Date
Page

Name: Kshitiz Reddy. M

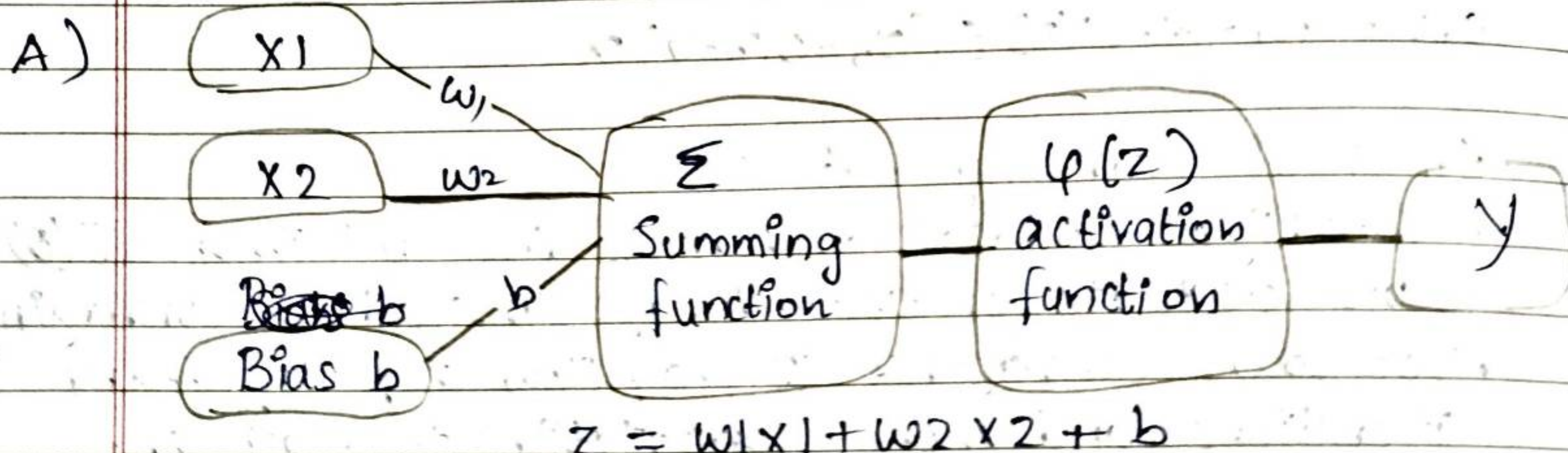
Reg No: A26MTA10008

Subject: Advanced Machine Learning

Part 1: Perceptrons and Basic Concepts

Q1) The perceptron Model

Draw a diagram of a single perceptron with two inputs (x_1, x_2), a bias term, and a step function activation. Label the weights (w_1, w_2), the bias (b), the summing function, and the activation function $\phi(z)$.



That's the perceptron: two inputs (x_1, x_2) and a bias each carry a weight into the summing function $z = w_1x_1 + w_2x_2 + b$, which then passes through the activation function $\phi(z)$ to produce output y .

Q2) Perceptron Computation

Consider a perceptron with weights $w_1 = 0.5$, $w_2 = -1.0$, and bias $b = 0.2$.

- Calculate the output z (the weighted sum) for the input vector $(x_1, x_2) = (1, -1)$.
- Calculate the final output y if the activation function is the ReLU function ($\text{ReLU}(z) = \max(0, z)$).
- Calculate the final output y if the activation function is $\text{Step}(z) = 1$ if $z \geq 0$, else 0 .

A) $w_1 = 0.5$, $w_2 = -1.0$, $b = 0.2$, Input $(x_1, x_2) = (1, -1)$

a) $z = w_1 x_1 + w_2 x_2 + b = (0.5)(1) + (-1.0)(-1) + 0.2$
 $= 0.5 + 1.0 + 0.2$

$z = 1.7$

b) $\text{ReLU}(z) = \max(0, 1.7) = 1.7$

c) $\text{Step}(z) = 1$ since $z = 1.7 > 0$, so $y = 1$

03) The Role of Bias

A perceptron without a bias term is trained to classify points. The decision boundary is $2x_1 + x_2 = 0$.

a) Plot this decision boundary. Shade the region where the perceptron would output 1.

b) Show that this perceptron cannot learn the simple AND function, whose outputs are $(0,0) \rightarrow 0$, $(0,1) \rightarrow 0$, $(1,0) \rightarrow 0$, $(1,1) \rightarrow 1$, by finding a counterexample.

c) Now, propose a new decision boundary of the form $w_1x_1 + w_2x_2 + b = 0$ that can correctly model the AND function. State the values for w_1 , w_2 & b .

A) a) The line $2x_1 + x_2 = 0$ passes through the origin. The region where $2x_1 + x_2 > 0$ (above/right of the line) is where the perceptron outputs 1.

b) Counterexample for AND: Test input $(0,1)$, which AND requires to output 0. $2(0) + 1 = 1$, and since $1 > 0$, the perceptron outputs 1 - but AND expects 0. This is a contradiction, so a boundary through the origin cannot separate AND's classes (AND is not linearly separable through the origin, since $(0,0) \rightarrow 0$ forces the boundary through the origin, but $(1,0) \rightarrow 0$ and $(0,1) \rightarrow 0$ while $(1,1) \rightarrow 1$ requires a boundary that excludes the axes).

c) A working boundary : $w_1 = 1, w_2 = 1, b = -1.5$, giving
decision rule $x_1 + x_2 - 1.5 \geq 0$

Check against AND:

- (0,0): $0+0-1.5 = -1.5 < 0 \rightarrow 0 \checkmark$
- (0,1): $0+1-1.5 = -0.5 < 0 \rightarrow 0 \checkmark$
- (1,0): $1+0-1.5 = -0.5 < 0 \rightarrow 0 \checkmark$
- (1,1): $1+1-1.5 = 0.5 \geq 0 \rightarrow 1 \checkmark$

Q4) Activation Functions

Explain the primary purpose of an activation function in a neural network. Then, state one significant advantage and one significant disadvantage of each following function:

- a) Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$
b) Hyperbolic Tangent: $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$
c) Rectified Linear Unit (ReLU): $\text{ReLU}(z) = \max(0, z)$

A) Purpose: Activation functions introduce non-linearity into the network. Without them, no matter how many layers you stack, the whole network collapses to a single linear transformation (a composition of linear functions is still linear), which can't model complex curved decision boundaries or relationships found in real data.

a) Sigmoid $\sigma(z) = 1 / (1 + e^{-z})$

- Advantage: Output is bounded in $(0, 1)$, which makes it interpretable as a probability - useful for output layers in binary classification.
- Disadvantage: Saturates for large $|z|$, causing near-zero gradients (vanishing gradient problem), and its output isn't zero-centered, which slows convergence.

b) Tanh $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$

- Advantage: Zero-centered output in $(-1, 1)$, so gradients flow more symmetrically, generally

leading to faster convergence than sigmoid.

- Disadvantages: Still saturates at extreme values, so it still suffers from vanishing gradients in deep networks.

c) ReLU $\text{ReLU}(z) = \max(0, z)$

- Advantage: Computationally cheap and avoids vanishing gradients for positive inputs, which speeds up training significantly in deep networks.

- Disadvantages: Neurons can "die" - if a neuron's input is consistently negative, its gradient is permanently zero and it stops learning entirely.

05) Creating a Logic gate

Design a perceptron (specify weights w_1, w_2 and bias b) that implements the logical OR function. Use the unit step function as the activation. Confirm your design works by calculating the output for all four possible inputs: $(0,0), (0,1), (1,0), (1,1)$.

A) Design: $w_1=1, w_2=1, b=-0.5$, step activation

- $(0,0): 0+0-0.5 = -0.5 < 0 \rightarrow 0 \checkmark$
- $(0,1): 0+1-0.5 = 0.5 \geq 0 \rightarrow 1 \checkmark$
- $(1,0): 1+0-0.5 = 0.5 \geq 0 \rightarrow 1 \checkmark$
- $(1,1): 1+1-0.5 = 1.5 \geq 0 \rightarrow 1 \checkmark$

Matches the OR truth table exactly.

Part 2: Backpropagation: Network Architecture and Initial Parameters

Forward Pass

Hidden layer:

$$\begin{aligned} \bullet \text{net H1} &= (0.05)(0.15) + (0.10)(0.20) + 0.35 = 0.3175 \\ &\rightarrow \text{out H1} = \text{Sigmoid}(0.3175) = 0.593270 \\ \bullet \text{net H2} &= (0.05)(0.25) + (0.10)(0.30) + 0.35 = 0.3925 \\ &\rightarrow \text{out H2} = \text{Sigmoid}(0.3925) = 0.596884 \end{aligned}$$

Output layer:

$$\begin{aligned} \bullet \text{net y1} &= (0.593270)(0.40) + (0.596884)(0.50) + 0.60 \\ &= 1.135750 \\ &\rightarrow \text{out y1} = \text{Sigmoid}(1.135750) = 0.751365 \\ \bullet \text{net y2} &= (0.593270)(0.45) + (0.596884)(0.55) + 0.60 \\ &= 1.195258 \\ &\rightarrow \text{out y2} = \text{Sigmoid}(1.195258) = 0.772928 \end{aligned}$$

Total error:

$$\begin{aligned} E_{\text{total}} &= \frac{1}{2} (0.01 - 0.751365)^2 + \frac{1}{2} (0.99 - 0.772928)^2 \\ &= 0.274811 + 0.023560 \\ &= 0.298371 \end{aligned}$$

Backward Pass:

Output-layer deltas ($\delta = \partial E / \partial \text{net}$):

$$\begin{aligned} \bullet \delta_{o1} &= (\text{out y1} - 1) \cdot \text{out y1} \cdot (1 - \text{out y1}) \\ &= 0.751365 \times 0.751365 \times 0.248635 \\ &= 0.138499 \end{aligned}$$

Date _____
Page _____

$$\begin{aligned} \bullet \delta_{02} &= (\text{out}_2 - T_2) \cdot \text{out}_2 \cdot (1 - \text{out}_2) \\ &= (-0.214072) \times 0.772928 \times 0.227012 \\ &= -0.038098 \end{aligned}$$

w5-w8 gradients and updates ($\eta = 0.5$):

$$\begin{aligned} \bullet \delta E / \delta w_5 &= \delta_{01} \cdot \text{out}_{H1} = 0.138499 \times 0.593270 = 0.082167 \\ &\rightarrow w_{5\text{-new}} = 0.4 - 0.5(0.082167) = 0.358917 \\ \bullet \delta E / \delta w_6 &= \delta_{02} \cdot \text{out}_{H1} = -0.038098 \times 0.593270 = -0.022603 \\ &\rightarrow w_{6\text{-new}} = 0.45 - 0.5(-0.022603) = 0.461301 \\ \bullet \delta E / \delta w_7 &= \delta_{01} \cdot \text{out}_{H2} = 0.138499 \times 0.596884 = 0.082668 \\ &\rightarrow w_{7\text{-new}} = 0.50 - 0.5(0.082668) = 0.458666 \\ \bullet \delta E / \delta w_8 &= \delta_{02} \cdot \text{out}_{H2} = -0.038098 \times 0.596884 = -0.022740 \\ &\rightarrow w_{8\text{-new}} = 0.55 - 0.5(-0.022740) = 0.561370 \end{aligned}$$

Output bias b2:

$$\begin{aligned} \delta E / \delta b_2 &= \delta_{01} + \delta_{02} = 0.138499 - 0.038098 = 0.100401 \\ b_{2\text{-new}} &= 0.60 - 0.5(0.100401) = 0.549800 \end{aligned}$$

Hidden-layer deltas (using original w5-w8, not updates ones):

$$\begin{aligned} \bullet \delta E / \delta \text{out}_{H1} &= \delta_{01} \cdot w_5 + \delta_{02} \cdot w_6 \\ &= (0.138499)(0.40) + (-0.038098)(0.45) \\ &= 0.038255 \\ \bullet \delta E / \delta \text{out}_{H2} &= \delta_{01} \cdot w_7 + \delta_{02} \cdot w_8 \\ &= (0.138499)(0.50) + (-0.038098)(0.55) \\ &= 0.0048295 \end{aligned}$$

$$\begin{aligned} \delta_{H1} &= 0.038255 \times [\text{out}_{H1} (1 - \text{out}_{H1}) = 0.241301] = 0.009231 \\ \delta_{H2} &= 0.0048295 \times [\text{out}_{H2} (1 - \text{out}_{H2}) = 0.240613] = 0.011620 \end{aligned}$$

w_1 - w_4 gradients and updates:

- $\partial E / \partial w_1 = \delta H_1 \cdot x_1 = 0.009231 \times 0.05 = 0.000462$
 $\rightarrow w_{1_new} = 0.15 - 0.5(0.000462) = 0.149769$
- $\partial E / \partial w_2 = \delta H_1 \cdot x_2 = 0.009231 \times 0.10 = 0.000923$
 $\rightarrow w_{2_new} = 0.20 - 0.5(0.000923) = 0.199538$
- $\partial E / \partial w_3 = \delta H_2 \cdot x_1 = 0.011620 \times 0.05 = 0.000581$
 $\rightarrow w_{3_new} = 0.25 - 0.5(0.000581) = 0.249709$
- $\partial E / \partial w_4 = \delta H_2 \cdot x_2 = 0.011620 \times 0.10 = 0.001162$
 $\rightarrow w_{4_new} = 0.30 - 0.5(0.001162) = 0.299419$

Hidden bias b_1 :

$$\partial E / \partial b_1 = \delta H_1 + \delta H_2 = 0.009231 + 0.011620 = 0.020852$$

$$b_{1_new} = 0.35 - 0.5(0.020852) = 0.339574$$

Second Training cycle.

a) Recalculated hidden outputs with updated weights:

- $net H_{1_new} = (0.05)(0.149769) + (0.10)(0.199538) + 0.339574$
 $= 0.367016$
 $\rightarrow out H_{1_new} = 0.596699$
- $net H_{2_new} = (0.05)(0.249709) + (0.10)(0.299419) + 0.339574$
 $= 0.382001$
 $\rightarrow out H_{2_new} = 0.594298$

b) Recalculated outputs:

- $net y_1_new = 1.039446 \rightarrow out y_1_new = 0.737827$
- $net y_2_new = 1.155996 \rightarrow out y_2_new = 0.760573$

c) New total error:

$$\begin{aligned} E_{\text{new-total}} &= \frac{1}{2} (0.01 - 0.737827)^2 + \frac{1}{2} (0.99 - 0.760273)^2 \\ &= 0.264867 + 0.026319 \\ &= 0.291186 \end{aligned}$$

Q8) Analyzing one Epoch

a) Yes - E_{total} decreased from 0.29837 to ≈ 0.291186 , a small but real reduction.

b) Interestingly, the two outputs moved differently: out_1 moved closer to τ_1 ($0.751 \rightarrow 0.738$, target 0.01 - still far, but improving), while out_2 actually moved slightly further from τ_2 . This is normal: gradient descent minimizes the total error, not each output independently, and the two outputs are coupled through shared hidden-layer weights. One update is a small step in the direction that reduces the combined loss - it doesn't guarantee uniform improvement everywhere. This is exactly why training needs many epochs: repeated small steps gradually bring both outputs towards their targets, even if any single step looks mixed.

Q9) Vanishing Gradient Analysis

a) $\delta_{out H1} = out H1 (1 - out H1) = 0.593270 \times 0.406703$
 ≈ 0.241301

b) The sigmoid derivative peaks at 0.25 (at $z=0$) and shrinks towards 0 as $|z|$ grows in either direction. During backpropagation through a deep network, the gradient reaching an early layer is a product of many such derivatives terms, each ≤ 0.25 . Multiplying many numbers that are all less than 1 - often much less - causes the gradient magnitude to shrink exponentially with depth. By the time the gradient reaches the earliest layers, it can become vanishingly small, so those layers barely update and effectively stop learning. This is the vanishing gradient problem, and it worsens as networks get deeper.

Q10) Exploding Gradient Scenario

Weights all initialized to 5.0:

a) net $H1 = x1.w1 + x2.w2 + b1$ would jump to

$$(0.05)(5) + (0.10)(5) + 0.35 = 0.25 + 0.5 + 0.35 = 1.10$$

for single layer - and in a deeper network, with large weights compounding across layers and larger activations feeding forward, net values would grow rapidly, potentially reaching very large magnitudes.

b) With net $H1$ pushed far from zero, the sigmoid saturates, and its derivative out $H1$ approaches 0. So paradoxically, at the neuron level, large weights can also trigger vanishing gradients locally due to saturation.

c) The broader "exploding gradient" risk comes from the weights themselves: when backpropagating through many layers, gradients are repeatedly multiplied by these large weight values. If weights are consistently > 1 , the gradient magnitude can grow exponentially with depth instead of shrinking. This causes extremely large weight updates, wild oscillations in the loss, numerical overflow, and a training process that diverges instead of converging. The network fails to learn a stable set of weights.

811) Overfitting

a) With 10 trainable parameters and only 1 training sample, the model has vastly more degrees of freedom than the data can constrain. A single data point provides very little information about the true underlying relationship, but 10 free parameters give the network more than enough flexibility to drive the error on that one sample arbitrarily close to zero - essentially memorizing the exact input-output pair rather than learning any generalizable pattern.

b) After many iterations, the network would converge to $out_1 \approx 0.01$ and $out_2 \approx 0.99$ almost exactly - a near-perfect fit to this one example. But this "success" is illusory: the weights have been tuned specifically to this single input.

Q12) Convergence and local Minima

a) The goal of multiple epochs is to progressively adjust the weights so that the total error is minimized across the entire training set, guiding the network toward parameters that ~~only~~ not only fit the training data but also generalize to new, unseen inputs.

b) A local minimum is a point in the loss landscape where the error is lower than all immediately neighbouring points, but not necessarily the lowest possible error anywhere in the landscape. The global minimum is the single point where the error is truly the lowest achievable across the entire parameter space.

c) Finding the global minimum is hard because the loss surface of a neural network is high-dimensional and non-convex - riddled with many local minima, saddle points, and flat plateaus. Gradient descent only has access to local gradient information, so it has no way to "see" the whole landscape and can easily settle into a local minimum or stall on a saddle point / plateau where gradients are near zero.